

Assignment 2 Pull-Request

Jonathan Distler

September 17th, 2026

Sketches

The following are my sketches intended to represent key physical attributes of the setup. The first failure mode listed (pendulum parallel to the plane), isn't a true failure mode in our setup considering α is bounded, but it provides a more realistic failure mode in the case that the plane is frictionless and the leg extends to the point in which the waist torque can't provide enough torque input to drive the legs back up.

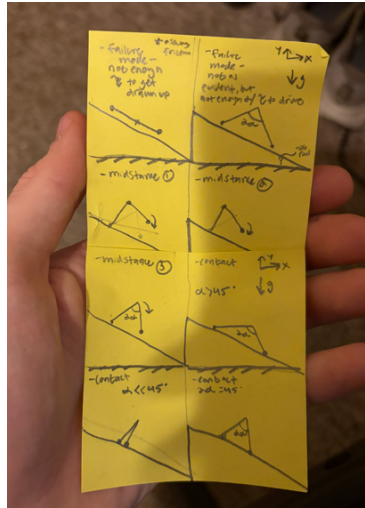


Figure 1: Sketches representing variations in failure modes, stances, and contacts

Visualization of RoA for ankle-controller

The figure below represents the Region-of-Attraction (RoA) of the state $(\theta, \dot{\theta})$.

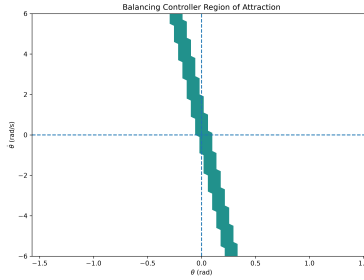


Figure 2: Visualization for the Region-of-Attraction given an ankle-torque controller

Choice of *Poincare'* section

Unlike assignment 1 where the *Poincare'* section was defined as the instantaneous moment the forward foot hit the ground, the new *Poincare'* section is defined as the vertical position between the two legs. This proved to be an intuitive section because the upright position is defined as the point in which the plane is orthogonal to the slope of the incline (γ). Additionally, it can be posited that the angular velocity is the $\Delta\theta$ across the plane (given a discrete time), thus $\dot{\theta}$ is positive when crossing the plane.

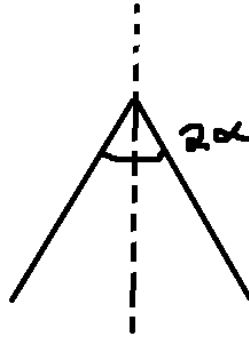


Figure 3: Representation of the *Poincare'* Section, Shown with the Dotted Line

Verification of Grid Solution

I chose an 81×21 grid size for the final solution. I iterated over $\dot{\theta}$ and α values in the code and examined the number of backward-iteration steps required for the RoA to converge. I varied the $\dot{\theta}$ grid size from 11 to 81 points and the α grid size from 11 to 31 points. I assumed that $\dot{\theta}$ and α could be treated

independently, reducing the computational cost compared to iterating over every possible combination of the two variables. The results are shown below, with the + indicating how much the number of states changes from a baseline 1 x 1 gride size.

```

THETA_DOT GRID SIZE = 11 x 21
Backward iteration 1: 8 reachable states (+3)
Backward iteration 2: 11 reachable states (+3)
Backward iteration 3: 11 reachable states (+0)

THETA_DOT GRID SIZE = 21 x 21
Backward iteration 1: 15 reachable states (+6)
Backward iteration 2: 21 reachable states (+6)
Backward iteration 3: 21 reachable states (+0)

THETA_DOT GRID SIZE = 31 x 21
Backward iteration 1: 23 reachable states (+9)
Backward iteration 2: 31 reachable states (+8)
Backward iteration 3: 31 reachable states (+0)

THETA_DOT GRID SIZE = 41 x 21
Backward iteration 1: 30 reachable states (+12)
Backward iteration 2: 41 reachable states (+11)

THETA_DOT GRID SIZE = 61 x 21
Backward iteration 1: 46 reachable states (+19)
Backward iteration 2: 61 reachable states (+15)

THETA_DOT GRID SIZE = 81 x 21
Backward iteration 1: 61 reachable states (+25)
Backward iteration 2: 81 reachable states (+20)

ALPHA GRID SIZE = 41 x 11
Backward iteration 1: 30 reachable states (+12)
Backward iteration 2: 41 reachable states (+11)

ALPHA GRID SIZE = 41 x 16
Backward iteration 1: 30 reachable states (+12)
Backward iteration 2: 41 reachable states (+11)

ALPHA GRID SIZE = 41 x 21
Backward iteration 1: 30 reachable states (+12)
Backward iteration 2: 41 reachable states (+11)
Backward iteration 3: 41 reachable states (+0)

ALPHA GRID SIZE = 41 x 31

```

Backward iteration 1: 30 reachable states (+12)
 Backward iteration 2: 41 reachable states (+11)
 Backward iteration 3: 41 reachable states (+0)

Trajectory for IC requiring at least 3 steps - plot maximum number of steps the walker can continue walking with before RoA

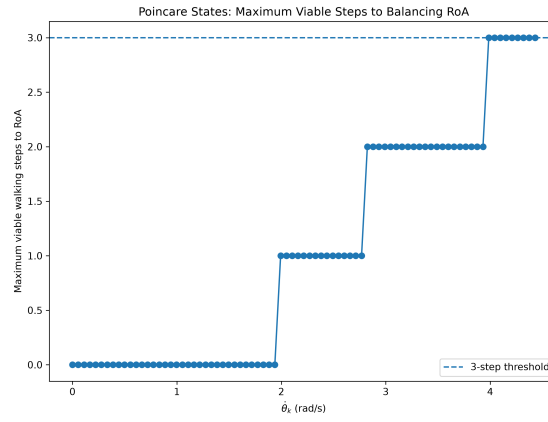


Figure 4: Maximum Steps to RoA

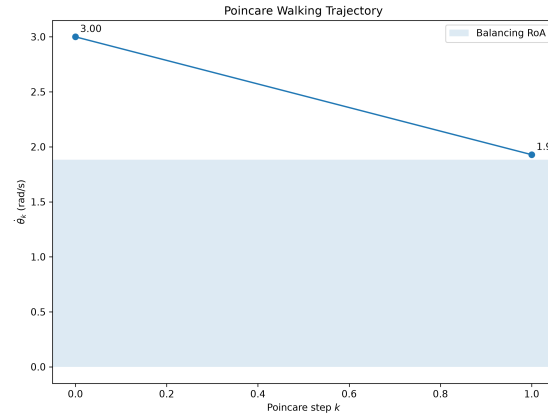


Figure 5: Walking *Poincare* Trajectory

Visualization of number of steps to reach standstill given IC

The graph below represents the number of steps required to reach a standstill given an initial-condition ($\dot{\theta}$). The vertical line represents the number of the particular initial-condition used in the simulation.

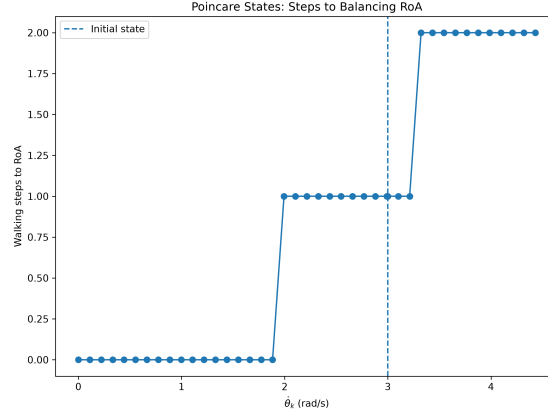


Figure 6: Visualization of the Minimal Number of Steps Needed to Reach a Standstill Given Initial-Conditions

Comments for Reviewer

- The biggest thing is reviewing the verification of grid solution. I assumed the two parameters were independent, to save time on computation, but do you think it is worthwhile parsing over every single permutation and having more elements in each set? Also, how would one define the number of elements in each set appropriately?
- I had to use the `.copy()` command a lot in my code, are there any alternatives that create local changes to arrays, as `.copy()` doesn't scale very well, without overwriting the global array scope? Or, is this more of an architecture design issue?
- I think that my RoA test seems to naive, but I can't intuit why? Are there any additional checks for falling in the RoA aside from requiring too much ankle torque?
- Is it reasonable to discard every 50 frames when creating the gif representation of the walking motion? I only added one in every 50 frames and it sped up computation, but I'm concerned this might neglect helpful intuition.

- Is it worthwhile to include the dynamic time-step as was commonly implemented in the assignment 1? I recognized that the state space we are brute-forcing over is orders of magnitude smaller than the last assignment, so the small time saving is pretty negligible.